



Reference guide: Lists

You've been learning that lists are important data structures in Python. A list is a data structure that helps store and manipulate an ordered collection of items. These items can be of any data type such as integers, floats, strings, and even other lists. Because they are so versatile, data professionals and all Python programmers use lists every day, so it's important to be familiar with how they work. This reading is a reference guide for lists designed to help you as you learn Python.

Create a list

There are two main ways to create lists in Python:

- Square brackets: `[]`
- The list function: `list()`

When instantiating a list using brackets, separate each element with a comma. For example, the following code creates a list of strings:

```
list_a = ['olive', 'palm', 'coconut']
```

Result: `['olive', 'palm', 'coconut']`

You can also create a list of integers:

```
list_b = [8, 6, 7, 5, 3, 0, 8]
```

Result: `[8, 6, 7, 5, 3, 0, 8]`

Or a list of mixed data types:

```
list_c = ['Abidjan', 14.2, [1, 2, None], 'Zagreb']
```

Result: `['Abidjan', 14.2, [1, 2, None], 'Zagreb']`

To create an empty list, use empty brackets or the `list()` function:

```
empty_list_1 = []
```

```
empty_list_2 = list()
```

Result: `[]`

The `list()` function can also convert iterable data types to lists:

```
my_string = 'Wilson'
my_list = list(my_string)
print(my_list)
```

Result: ['W', 'i', 'l', 's', 'o', 'n']

Indexing and slicing

Just as with strings, you can access elements in a list using indexing and slicing. The first element of a list has index zero, the second element has index one, and so on. Use square brackets to index:

```
phrase = ['Astra', 'inclinant', 'sed', 'non', 'obligant']
print(phrase[1])
```

Result: inclinant

You can also use negative indices to access items from the end of a list:

```
phrase = ['Astra', 'inclinant', 'sed', 'non', 'obligant']
print(phrase[-1])
```

Result: obligant

Use slicing to extract a sublist. To slice, use square brackets containing a range of indices separated by a colon:

```
phrase = ['Astra', 'inclinant', 'sed', 'non', 'obligant']
print(phrase[1:4])
```

Result: ['inclinant', 'sed', 'non']

Notice that this code returned a sublist containing the elements at indices one, two, and three of phrase. The ending index of the slice is not included.

Omitting the starting index in a slice implies an index of zero, and omitting the ending index implies an index of `len(my_list)`:

```
phrase = ['Astra', 'inclinant', 'sed', 'non', 'obligant']
print(phrase[:3])
print(phrase[3:])
```

Result: ['Astra', 'inclinant', 'sed']

['non', 'obligant']

List mutability

Lists are mutable, which means that you can change their contents after they are created. You can change an individual item in a list by specifying its index and assigning a new value to it. For example:

```
my_list = ['Macduff', 'Malcolm', 'Duncan', 'Banquo']
my_list[2] = 'Macbeth'
print(my_list)
```

Result: ['Macduff', 'Malcolm', 'Macbeth', 'Banquo']

You can even change a slice of a list using the same logic. The slice can be of any length. The elements in the new list will be inserted in place of the indicated slice:

```
my_list = ['Macduff', 'Malcolm', 'Macbeth', 'Banquo']
my_list[1:3] = [1, 2, 3, 4]
print(my_list)
```

Result: ['Macduff', 1, 2, 3, 4, 'Banquo']

List operations

Lists can be combined using the addition operator (+):

```
num_list = [1, 2, 3]
char_list = ['a', 'b', 'c']
num_list + char_list
```

Result: [[1, 2, 3, 'a', 'b', 'c']]

They can also be multiplied using the multiplication operator (*):

```
list_a = ['a', 'b', 'c']
list_a * 2
```

Result: ['a', 'b', 'c', 'a', 'b', 'c']

But they cannot be subtracted or divided.

You can check whether a value is contained in a list by using the `in` operator:

```
num_list = [2, 4, 6]
print(5 in num_list)
print(5 not in num_list)
```

Result: False

True

List methods

Lists are a core Python class. As you've learned, classes package data together with tools to work with it. Methods are functions that belong to a class. Lists have a number of built-in methods that are very useful. These include:

append()

Add an element to the end of a list:

```
my_list = [0, 1, 1, 2, 3]
variable = 5
my_list.append(variable)
print(my_list)
```

Result: [0, 1, 1, 2, 3, 5]

insert()

Insert an element at a given position:

```
my_list = ['a', 'b', 'd']
my_list.insert(2, 'c')
print(my_list)
```

Result: ['a', 'b', 'c', 'd']

remove()

Remove the first occurrence of an item:

```
my_list = ['a', 'b', 'd', 'a']
my_list.remove('a')
print(my_list)
```

Result: ['b', 'd', 'a']

pop()

Remove the item at the given position in the list, and return it. If no index is specified, a.pop() removes and returns the last item in the list:

```
my_list = ['a', 'b', 'c']
my_list.pop()
print(my_list)
```

Result: c

```
['a', 'b']
```

clear()

Remove all items:

```
my_list = ['a', 'b', 'c']  
my_list.clear()  
print(my_list)
```

Result: []

index()

Return the index of the first occurrence of an item in the list:

```
my_list = ['a', 'b', 'c', 'a']  
my_list.index('a')
```

Result: 0

count()

Return the number of times an item occurs in the list:

```
my_list = ['a', 'b', 'c', 'a']  
my_list.count('a')
```

Result: 2

sort()

```
char_list = ['b', 'c', 'a']  
num_list = [2, 3, 1]  
char_list.sort()  
num_list.sort(reverse=True)  
print(char_list)  
print(num_list)
```

Result: ['a', 'b', 'c']

```
[3, 2, 1]
```

Key takeaways

Python lists are a versatile and essential data structure that store and manipulate ordered collections of various data types. Because lists are mutable, you can change, add, or remove elements after their creation using a variety of built-in methods. You can also access individual elements or sublists using indexing and slicing.

Additional resources

- For more information about lists, refer to [An Informal Introduction to Python: Lists](#).
- For more list methods, refer to [Data Structures: More on Lists](#).